Controlling Processes

Debian Linux: process identity, signals, and resource control

tsaimh (2022–2026, CC BY-SA)
wangth (2017–2021, CC BY-SA)
? (1996–2016)



What You Will Learn

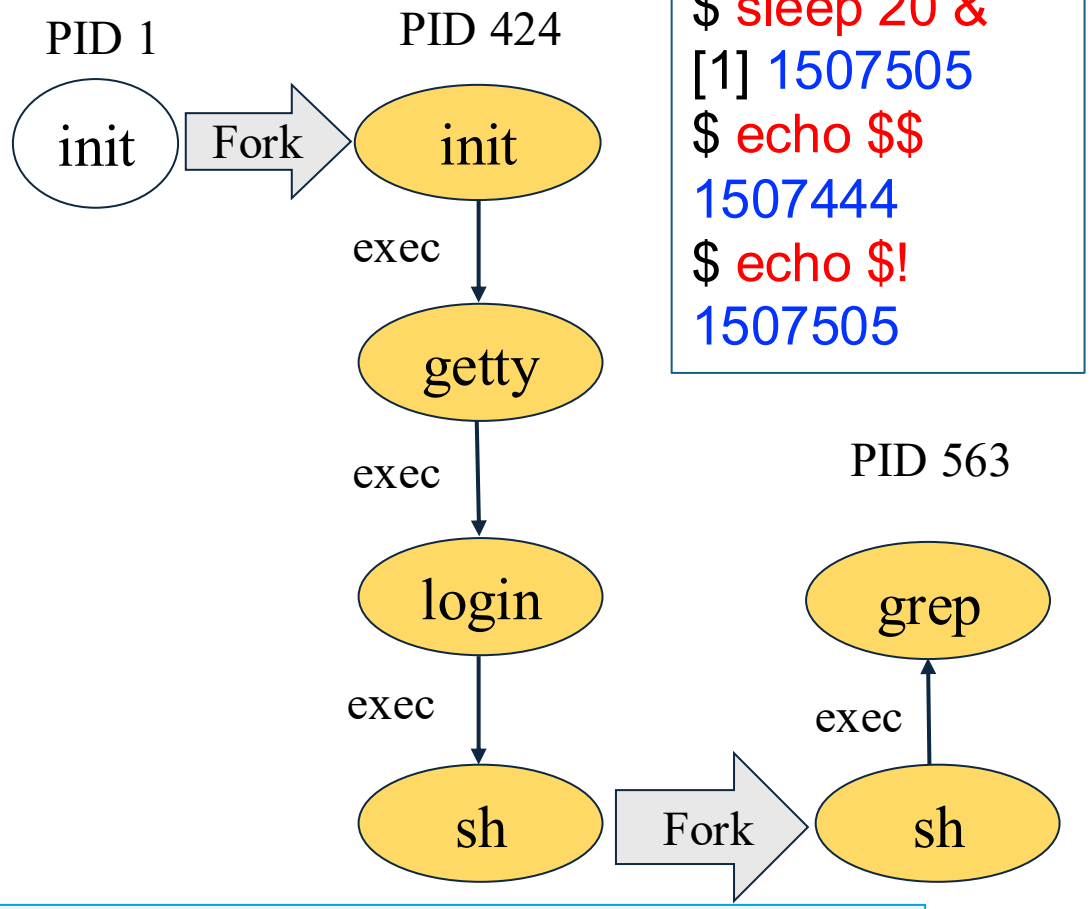
- Explain process identity, fork/exec, and parent–child relationships.
- Read Debian process states and identify resource use with ps and top.
- Use job control and signals to manage a known process.
- Adjust niceness and CPU affinity, then verify the result.
- Contain excessive process creation with systemd task limits.

1. Process Identity and Lifecycle

Program images, execution context, and parent–child relationships

Programs and Processes

Concept	Meaning	Example
Program	Executable code stored as a file	/usr/bin/sleep
Process	An executing program with its own context	sleep 300
PID	Identifier in a PID namespace	A value recorded in $$ or $!
Thread	Execution path within a process	TID / LWP on Linux




```
$ which sleep
/usr/bin/sleep
$ sleep 20 &
[1] 1507505
$ echo $$
1507444
$ echo $!
1507505
```

One program, several processes

Starting the same executable twice normally creates two distinct processes.

Process Execution Context

- Virtual address space contains code, data, heap, and stack mappings.
- Kernel records track credentials, scheduling state, and resource use.
- Open file descriptors connect the process to files, sockets, and pipes.
- The working directory, environment, and signal settings affect execution.

Inspection is a snapshot
Values can change while you inspect a running process.

PID, PPID, and Credentials

Field	Meaning
PID / PPID	Process ID / current parent process ID
RUID / EUID	Real user ID / effective user ID
RGID / EGID	Real group ID / effective group ID
Supplementary groups	Additional group memberships used in access checks
NI	Nice value used by normal scheduling policies

PID lifetime
PIDs can wrap and be reused. Verify the target again before sending a signal.

fork, exec, exit, and wait

Operation	Result
fork()	Creates a child with a new PID and copied process context
execve()	Replaces the program image while retaining the PID
_exit() / exit()	Terminates execution and makes exit status available
wait() / waitpid()	Collects a child status and reaps a terminated child

Process creation and replacement
fork creates a process. A successful exec replaces the program in that process.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main(){
  int pid, i;
  pid = fork();
  if(pid ==0) {
     for (int i=0; i<5; i++) {
        printf("I am child, pid is %d,”
         “ppid is %d\n", getpid(), getppid());
        sleep(1);
     }
     exit(1);
  }else if (pid >0) {
     for (int i=0; i<5; i++) {
         printf("I am parent, pid is %d,”
        “ppid is %d\n", getpid(),getppid());
         sleep(1);
     }
  }else if (pid < 0)
     printf("Something wrong while forking\n");
  return 0;
}
```

```
I am parent, pid is 1485, ppid is 1125
I am child, pid is 1486, ppid is 1485
I am parent, pid is 1485, ppid is 1125
I am child, pid is 1486, ppid is 1485
I am parent, pid is 1485, ppid is 1125
I am child, pid is 1486, ppid is 1485
I am child, pid is 1486, ppid is 1485
I am parent, pid is 1485, ppid is 1125
I am child, pid is 1486, ppid is 1485
I am parent, pid is 1485, ppid is 1125
```

Zombies and Orphans

Case	What happened	Management implication
Zombie (Z)	Child exited, parent has not reaped it	Investigate the parent and its wait logic
Orphan	Parent exited before the child	Linux reparents to a subreaper or namespace init
Stopped (T)	Execution paused by a stop signal	A continue signal can resume it

Zombie diagnosis
A zombie has already exited. Sending KILL cannot make it exit again.

Effective IDs and set-user-ID

```
$ id
uid=12134(tsaimh) gid=1010(faculty)
groups=1010(faculty),2000(taever)
$ ls -l /etc/passwd /etc/shadow
-rw-r--r-- 1 root root   1655 Aug  3 09:04 /etc/passwd
-rw-r----- 1 root shadow  866 Aug  3 09:04 /etc/shadow
$ ps -o pid,ruid,euid,rgid,egid,args -p $$
    PID  RUID  EUID  RGID  EGID COMMAND
1527071 12134 12134  1010  1010 bash
$ ls -al /usr/bin/passwd
-rwsr-xr-x   1 root  root  118168 Apr 19 /usr/bin/passwd
```

- Access checks can depend on effective credentials.
- A set-user-ID executable may change EUID to its owner on exec.
- nosuid, no_new_privs, and tracing can suppress that transition.
- Debian stores protected local password data in /etc/shadow.

Local policy matters

Inspect the installed executable and permissions before explaining its privilege path.

2. Process Observation

Debian process states, snapshots, and live monitoring

Linux Process States

State	Meaning
R	Running or runnable
S	Interruptible sleep
D	Uninterruptible sleep, often waiting for I/O
T / t	Stopped by job control / stopped by a debugger
Z	Exited child awaiting reaping
I	Idle kernel thread

A state is evidence
D does not prove a disk failure. Check the wait channel, kernel messages, and storage path.

ps: Selection and Output

```
$ ps auxww
USER         PID   %CPU %MEM    VSZ   RSS  TTY      STAT START  TIME COMMAND
tsaimh   1546761  200    0.0    20288  4920 pts/1    R+   16:14   0:00 ps auxww

$ ps -eo user,pid,ppid,stat,ni,pcpu,pmem,args --sort=-pcpu
USER         PID    PPID    STAT  NI %CPU %MEM COMMAND
tsaimh   1546580 1546369 R+   0  100   0.0     ps -eo user,pid,ppid,stat,ni,pcpu,pm
root         1007       1       Ssl    0  0.7    0.0     /usr/bin/crowdsec-firewall-bouncer -
prometh+    952       1       Ssl    0  0.3    0.0     /usr/bin/prometheus-node-exporter --

$ ps -o pid,ppid,stat,etime,time,args -p $$
 PID          PPID STAT     ELAPSED     TIME     COMMAND
1546369 1546368 Ss         11:05.  00:00:00   -bash
```

Debian procps-ng

ps aux uses BSD-style options without a dash.

Reading ps Fields

```
$ sleep 20 &
$ ps -o vsz,rss,pss,cmd
  VSZ   RSS   PSS CMD
22608  3724  1394 -bash
 5436   904   190 sleep 20
11200  4400  2198 ps -o vsz,rss,pss,cmd
```

Field	Interpretation
%CPU	CPU time divided by elapsed lifetime in ps
TIME / ELAPSED	Accumulated CPU time / wall-clock time since start
VSZ / RSS	Virtual address size / resident memory, reported in KiB
STAT	Base state followed by flags such as s, +, or l
ARGS	Command arguments, which may contain sensitive data

Memory interpretation

VSZ includes mappings and reservations. Shared pages can appear in several RSS totals.

top on Debian Linux

Key / command	Action
P / M / T	Sort by CPU use / resident memory / CPU time
1 / H	Toggle per-CPU display / thread display
u / c	Filter by user / toggle command line display
h / q	Show help / quit
top -p PID	Monitor a known process
top -b -n 1	Print one batch snapshot

Linux and FreeBSD differ
Debian top -u filters by user. Learn keys from h in the installed version.

CPU Use, Load, and Memory

- A process with several active threads can exceed 100% CPU in common top modes.
- Load averages include runnable tasks and tasks in uninterruptible sleep.
- Compare load with available CPUs and investigate waiting as well as execution.
- Check available memory and sustained swap activity before diagnosing pressure.

```
top - 21:47:30 up 12 days,  1:25,  3 users,  load average: 0.03, 0.19, 3.11
Tasks: 296 total,   1 running, 295 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.9 us,  0.7 sy,  0.0 ni, 98.3 id,  0.0 wa,  0.0 hi,  0.1 si,  0.0 st
```

A high number needs context

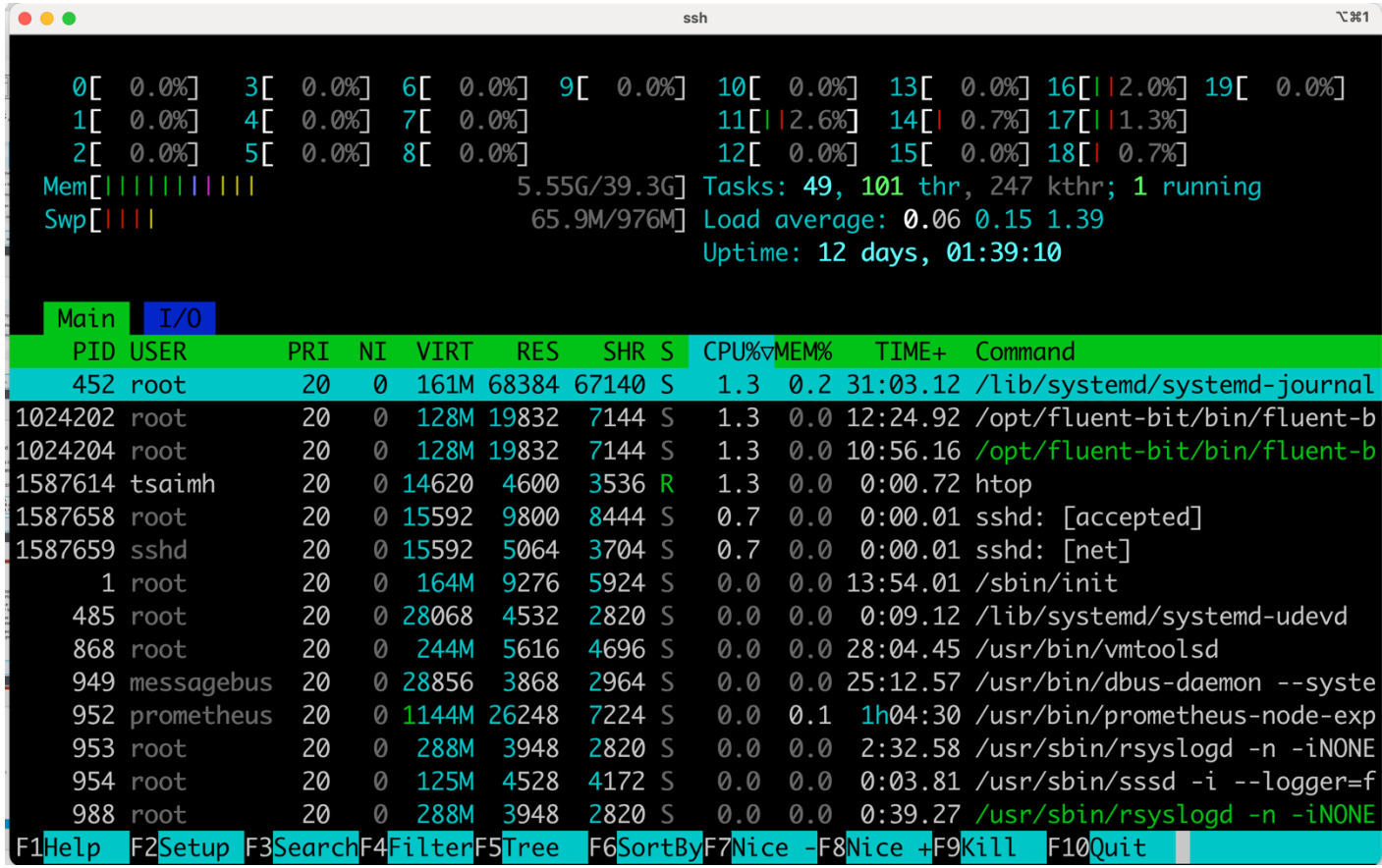
Load average is a task count average, rather than a CPU percentage.

htop and Supporting Tools

```
sudo apt install htop
htop

# Debian procps tools
free -h
vmstat 1
```

- htop provides interactive filtering, sorting, and tree views.
- Use the displayed key bar and help for local key bindings.
- free summarizes memory. vmstat shows repeated system samples.
- Process visibility and control still depend on permissions.



3. Signals and Job Control

Signal delivery, shell jobs, and controlled termination

Signal Sources and Actions

Source	Example
Terminal driver	Ctrl-C sends INT to the foreground process group
Shell job control	Ctrl-Z normally sends TSTP to that group
Another process	kill sends a selected signal, subject to permission checks
Kernel	Synchronous faults can generate SEGV or FPE

Signal disposition
A process can use a handler, ignore a signal, or retain its default action.

Blocked, Ignored, and Pending Signals

- Ignoring a signal chooses to discard it.
- Blocking a signal defers eligible delivery until the mask allows it.
- Standard signals do not queue repeated instances of the same signal.
- KILL and STOP cannot be caught, blocked, or ignored.

#	Name	Description	Default	Catch	Block
1	SIGHUP	Hangup	Terminate	✓	✓
2	SIGINT	Interrupt (^C)	Terminate	✓	✓
3	SIGQUIT	Quit	Terminate	✓	✓
9	SIGKILL	Kill	Terminate	✗	✗
10	SIGBUS	Bus error	Terminate	✓	✓
11	SIGSEGV	Segmentation fault	Terminate	✓	✓
15	SIGTERM	Soft. termination	Terminate	✓	✓
19	SIGSTOP	Stop	Stop	✗	✗
20	SIGTSTP	Stop from tty (^Z)	Stop	✓	✓
18	SIGCONT	Continue after stop	Ignore	✓	✗

Names are safer than numbers

Signal numbers can vary by platform and architecture.
Use kill -l and symbolic names.

Bash Job Control

```
# Run in an interactive Bash terminal
$ sleep 300
^Z
Suspended
$ jobs -l
[1]  + 1592397 Suspended                 sleep 300
$ bg %1
[1]    sleep 300 &
$ fg %1
sleep 300
^C
$
```

- jobs lists jobs known to the current shell.
- %1 is a shell job specification.
- bg resumes a stopped job in the background.
- fg brings it into the foreground.

Jobs and PIDs
A shell job can contain a pipeline with several processes.

Targeted Signals with kill

```
$ sleep 300 &
[1] 1594652
$ pid=$!
$ ps -o pid,ppid,stat,args -p "$pid"
    PID    PPID STAT COMMAND
1594652 1594651 S    sleep 300
$ kill -TERM "$pid"
$ wait "$pid"
[1]+  Terminated              sleep 300
$ ps -p "$pid"
    PID TTY          TIME CMD
$
```

- Capture $! immediately after starting the job.
- Verify the PID and command before sending the signal.
- TERM allows an application to run a handler.
- Confirm termination and interpret the exit status.

4. Scheduling and Resource Limits

CPU scheduling preferences, allowed CPUs, and task containment

Niceness on Linux

Property	Meaning
Normal range	-20 to 19. Higher NI means lower scheduling weight
Inheritance	Children inherit nice values across fork
Permission	Raising NI is usually allowed for your own tasks
Priority increase	Lowering NI normally requires privilege or a resource allowance
Scope	Applies to relevant normal scheduling policies

Niceness and CPU limits
Niceness changes competition for CPU time. It does not impose a fixed CPU percentage.

nice and renice

```
$ nice -n 10 sleep 300 &
[2] 1597304
$ pid=$!
$ ps -o pid,ni,pri,comm -p "$pid"
    PID     NI PRI COMMAND
1597304  10   9 sleep
$ renice --priority 15 --pid "$pid"
1597304 (process ID) old priority 10, new priority 15
$ ps -o pid,ni,pri,comm -p "$pid"
    PID     NI PRI COMMAND
1597304  15   4 sleep
$ renice --priority 20 --pid "$pid"
1597304 (process ID) old priority 15, new priority 19
$ ps -o pid,ni,pri,comm -p "$pid"
    PID     NI PRI COMMAND
1597304  19   0 sleep
$
```

- nice -n 10 adjusts the inherited value by +10.
- renice --priority 15 sets an absolute value.
- The example assumes an ordinary shell starting at NI 0.
- Check NI after each change.

CPU Affinity with taskset

```
# Query CPUs allowed for the current shell
$ taskset -pc $$
pid 1594651's current affinity list: 0-19

# Choose a CPU from that allowed list
$ cpu=5
$ taskset -c "$cpu" sleep 300 &
[1] 1598297
$ pid=$!
$ taskset -pc "$pid"
pid 1598297's current affinity list: 5
$ taskset -c "3,7-11" sleep 300 &
[2] 1598321
$ pid=$!
$ taskset -pc "$pid"
pid 1598321's current affinity list: 3,7-11
```

- Affinity restricts the CPUs a task may run on.
- It does not reserve a CPU or create parallelism.
- To change an existing task: taskset -pc LIST PID.
- For all existing threads, consider taskset -apc LIST PID.

Fork Bombs and Task Exhaustion

- Unbounded child creation can exhaust task slots and other resources.
- A shell or recovery command may fail because it cannot create a process.
- Repeated name-based killing can race with new children.
- A cgroup task limit bounds descendants within a managed unit.

```
last pid: 14928;  load averages: 53.07, 53.10, 53.08
210 processes: 55 running, 154 sleeping, 1 zombie
CPU:  0.0% user, 49.7% nice,  0.1% system,  0.0% interrupt, 50.1% idle
Mem: 38M Active, 760M Inact, 2904M Wired, 40K Cache, 255M Buf, 4220M Free
ARC: 2047M Total, 572M MFU, 897M MRU, 16K Anon, 16M Header, 562M Other
Swap: 4096M Total, 4096M Free

  PID USERNAME          THR PRI NICE   SIZE    RES STATE   C   TIME    WCPU COMMAND
 4224 [redacted]          1  97   20 19760K  2924K RUN    11  65:04  16.70% fork1
 4241                     1  96   20 19760K  2924K RUN     8  64:37  16.06% fork1
 4220                     1  96   20 19760K  2924K RUN     8  65:05  15.97% fork1
 6332                     1  96   20 19760K  2924K RUN    10 105:20  15.87% fork1
 4087                     1  96   20 19760K  2924K RUN    11  66:08  15.87% fork1
 4054                     1  96   20 19760K  2924K RUN    15  67:43  15.67% fork1
 4086                     1  96   20 19760K  2924K RUN    10  66:30  15.67% fork1
 6329                     1  96   20 19760K  2924K RUN    13 105:17  15.58% fork1
 4090                     1  96   20 19760K  2924K RUN    12  66:28  15.58% fork1
 4244                     1  96   20 19760K  2924K RUN    13  64:51  15.58% fork1
 4001                     1  96   20 19760K  2924K RUN    13  68:11  15.48% fork1
 4084                     1  96   20 19760K  2924K CPU13  13  66:24  15.48% fork1
 4242                     1  96   20 19760K  2924K RUN    13  65:04  15.48% fork1
 4225                     1  96   20 19760K  2924K RUN     9  65:00  15.48% fork1
 4221                     1  96   20 19760K  2924K RUN    11  64:52  15.48% fork1
 4243                     1  96   20 19760K  2924K RUN     8  64:48  15.48% fork1
```

How to Create a Fork Bomb

- C/C++

```c
#include <unistd.h>

int main(void) {
    while(1)
        fork();
    return 0;
}
```

- Perl

```
fork while fork
```

- Windows

```
%0 | %0
```

- Bash (Shell script)

```
:(){ :|:& };:
```

```
# Define function
forkbomb() {
    # Run twice with pipe
    forkbomb | forkbomb &
}
;
# Start the fork bomb
forkbomb
```

DON'T DO THAT!!!!

How to deal with fork bomb

- Just kill all of them

  $ killall -KILL bombName

- When you have no more resource to fork your shell

  $ exec killall -KILL bombName

  (The shell will become "killall", and never goes back.)

- "killall" isn’t an atomic command
  - More bombs may be created when killing them
  - Run multiple "killall"

A Bounded systemd Task Limit

```
# Use an existing user manager in the lab VM
$ systemd-run --user --unit=process-lab \
  -p TasksMax=32 /usr/bin/sleep 300
Running as unit: process-lab.service
$ systemctl --user show process-lab.service -p
TasksCurrent -p TasksMax
TasksCurrent=1
TasksMax=32
$ systemctl --user stop process-lab.service
$ systemctl --user show process-lab.service -p
TasksCurrent -p TasksMax
TasksCurrent=[not set]
TasksMax=48097
```

- TasksMax counts tasks, including threads.
- The limit applies to the service cgroup and descendants.
- The example starts one sleep process.
- Set limits before launching an untrusted workload.

Process Control Cheat Sheet

Task	Debian command
Inspect selected fields	ps -eo pid,ppid,stat,ni,args
Monitor live activity	top / htop
Signal a known process	kill -TERM PID
Start at a lower CPU preference	nice -n 10 COMMAND
Set an absolute nice value	renice --priority 15 --pid PID
Inspect allowed CPUs	taskset -pc PID

Final check
Verify the target, scope, and observed result for every management command.